



East West University
Department of CSE

Course Code: CSE 347

Course Title: Information System Analysis and Design

Project: StudyByte, Peer-to-Peer Micro-Tutoring

Task: Final Report on Project Closure

Semester and Year:
Spring 2026

GROUP NO: 04

Name of Student	Student ID
Belal Hossain	2023-2-60-010
Esrat Jahan Samiya	2023-2-60-002
Md. Majharul Islam	2023-3-60-127
Md. Jakaria Masud	2023-3-60-386

Course Instructor information:

Dr. Hosney Jahan
Assistant Professor
(Department of CSE)
East West University, Dhaka

Date of Submission:
22/04/2026

Acknowledgment

We are highly indebted for getting such a tremendous opportunity to prepare the report on the closure of StudyByte . It was a great honor for us to work on this project under the supervision of our amazing course instructor, Dr.Hosney Jahan, Assistant Professor, Department of Computer Science and Engineering, East West University, and we wholeheartedly thank her for providing us with valuable guidance and insights on how to prepare this report. Her constructive feedback and encouragement have been pivotal in completing this project. We are also sincerely grateful to all the individuals and stakeholders who provided their requirements and shared their insights, enabling us to develop a system that addresses practical needs effectively.

Abstract

StudyByte is a peer to peer online learning platform where students are able to log in as tutor or learner using their university mail. The tutor needs to submit his or her grade report during the sign up process, and after the approval from the admin the tutor can enlist topics based on his/her expertise, make a schedule and set the price (tokens that can be converted into real currency). The learner can see the courses enlisted on the market-place and buy them using required tokens for a specific schedule. A video session is held between the tutor and the learner, and only after the session ends, does the tutor get paid (though the learner pays during the purchase and the system freezes it till the session ends). A tutor can withdraw his earnings through BKash after the earnings reach a certain threshold, or can

spend it here from the Learner mode, buying other courses. A learner can give reviews about the tutor based on his experience during the session which will reflect on the tutor profile.

StudyByte

1. Introduction

1.1 Project Overview

StudyByte is a web-based Peer-to-Peer Micro-Tutoring Information System developed for university students. The system enables students to offer and receive topic-based tutoring sessions through a structured digital marketplace.

The platform integrates role-based access, booking and scheduling, a digital wallet system, escrow payment mechanism, session verification, and administrative monitoring to ensure security, trust, and quality learning.

1.2 Project Objectives

- To create a secure academic tutoring marketplace.
- To enable structured booking and scheduling.
- To implement a token-based wallet with escrow protection.
- To ensure verified session completion before payment release.
- To maintain tutor quality using rating and review systems.
- To provide administrative control and dispute management.

1.3 Project Scope

In Scope

- User registration and authentication
- Role-based dashboards
- Topic marketplace and search
- Booking and scheduling
- Digital wallet and escrow system
- Session verification
- Rating and review
- Admin monitoring

Out of Scope

- Built-in video conferencing
- External banking gateway integration
- Multi-institution deployment (Phase 1 limited to one university)

2. Core Features and Project Flow

2.1 Core Features

1. User Registration & Authentication
2. Role-Based Access Control
3. Profile Management
4. Topic Marketplace
5. Search & Filtering System
6. Availability & Scheduling System
7. Booking Management System
8. Digital Wallet System
9. Escrow Payment System
10. Session Verification System
11. Rating & Review System
12. Admin Control & Dispute Management

2.2 Logical Project Flow

➤ User Entry & Authentication

User: Student (Learner/Tutor), Admin

Logical Steps:

1. User registers using university email.
2. The system verifies the email domain.
3. User selects role (Learner or Tutor).
4. The system creates a profile.
5. Dashboard loads according to role.

Output: Authorized user enters system.

➤ Tutor Marketplace Setup

User: Tutor

Logical Steps:

1. Tutor logs in.
2. Tutor creates micro-topic listings.
3. Tutor sets:
 - o Token price
 - o Available time slots
4. System stores topic in marketplace database.
5. Topic becomes searchable.

Output: Marketplace updated with tutoring service.

➤ Learner Search & Booking

User: Learner

Logical Steps:

1. The learner searches for a topic.
 2. System filters by:
 - o Price
 - o Rating
 - o Urgency
 3. The learner selects a tutor.
 4. Learners select available time slots.
 5. The system checks wallet balance.
 6. Booking request confirmed.
- Output: Booking created.

➤ **Escrow Payment Protection**

Logical Steps:

1. Tokens deducted from the learner wallet.
2. Tokens moved to an escrow account.
3. Booking status updated to "Confirmed".

Important:

The tutor does NOT receive payment yet.

Output: Secure transaction locked.

➤ **Session Execution & Verification**

Logical Steps:

1. Session takes place.
2. The system generates unique meeting code.
3. Learner provides code to tutor.
4. Tutor submits code.

5. System verifies code.
6. If valid:
 - o Escrow releases tokens to tutor wallets.
 - o Booking marked as "Completed".

Output: Payment transferred securely.

➤ **Rating & Admin Oversight**

Logical Steps:

1. Learner submits rating & review.
2. System updates tutor average rating.
3. Admin monitors:
 - o Transactions
 - o Disputes
 - o Withdrawal requests
4. Admin can suspend accounts if necessary.

Output: Trust & governance maintained.

3. Requirement Determination

Functional Requirements

1. User Management

- 1.1 The system must allow students to register using university email.
- 1.2 The system must verify the email domain before account activation.
- 1.3 The system must allow secure login and logout.
- 1.4 The system must support three roles: Learner, Tutor, and Admin.
- 1.5 The system must provide separate dashboards for each role.

2. Profile Management

- 2.1 The system must allow users to update profile information.
- 2.2 The system must allow tutors to add expertise topics.
- 2.3 The system must store department and student ID information.

3. Marketplace Management

- 3.1 The system must allow tutors to list micro-topics.
- 3.2 The system must allow tutors to set a token price per session.
- 3.3 The system must organize topics by department/category.
- 3.4 The system must allow learners to search topics.
- 3.5 The system must allow filtering by price, rating, and urgency.

4. Booking & Scheduling

- 4.1 The system must allow tutors to set available time slots.
- 4.2 The system must prevent double booking of time slots.
- 4.3 The system must allow learners to book available sessions.
- 4.4 The system must store booking history.

5. Payment & Wallet System

- 5.1 The system must maintain a digital wallet for each user.
- 5.2 The system must deduct tokens upon booking.
- 5.3 The system must hold tokens in escrow until session verification.
- 5.4 The system must release tokens after meeting code validation.
- 5.5 The system must maintain transaction history.

6. Session Verification

- 6.1 The system must generate a unique meeting code per session.
- 6.2 The system must verify meeting code before releasing payment.

7. Rating & Review

- 7.1 The system must allow learners to rate tutors.
- 7.2 The system must calculate the average rating automatically.
- 7.3 The system must display rating in tutor profile.

8. Admin Control

- 8.1 The system must allow admin to monitor users.
- 8.2 The system must allow the admin to manage disputes.
- 8.3 The system must allow admin to approve token withdrawals.
- 8.4 The system must allow the admin to suspend suspicious accounts.

Non-Functional Requirements

1. Performance Requirements

- 1.1 The system must load pages within 2 seconds.
- 1.2 The system must support at least 500 concurrent users.
- 1.3 Wallet transactions must be processed within 3 seconds.

2. Security Requirements

- 2.1 Only registered users can access the system.
- 2.2 Only learners can book sessions.
- 2.3 Only tutors can list topics.
- 2.4 Only the admin can approve withdrawals.
- 2.5 Passwords must be encrypted.

3. Operational Requirements

- 3.1 The system must run on modern web browsers.
- 3.2 The system must support mobile responsiveness.
- 3.3 The system must maintain daily database backup.

4. Reliability Requirements

- 4.1 The system must ensure 99% uptime.
- 4.2 The escrow system must prevent payment loss.

5. Usability Requirements

- 5.1 The interface must be simple and user-friendly.
- 5.2 The booking process must not exceed five steps.
- 5.3 Error messages must be clear and understandable.

4. Feasibility Study

4.1 Technical Feasibility

- Development tools are HTML, CSS, Javascript, MySQL, PHP/Python and these are readily available.
- Any standard web server is sufficient for the hardware requirement.\
- System complexity is moderate and manageable.

Conclusion: Technically feasible.

4.2 Economic Feasibility

Cost Element	Cost (BDT)
Development Cost	40000 to 50000
Infrastructure Cost	60000 to 80000
Maintenance Cost	50000 to 60000

Tools and Softwares	0 (open source softwares)
Total cost	150000 to 180000

Conclusion: Economically feasible.

4.3 Operational Feasibility

- High acceptance among student population
- Simple interface provides ease of use
- Very suitable for university campus

Conclusion: Operationally feasible.

4.4 Schedule Feasibility

Activity	Time Required
Requirement Analysis	1 Week
System Design	1 Week
Database Design	2 week
User Module	1.5 week
Booking Module	1.5 week
Wallet & Escrow	2 week
Rating & Admin	1 week
Testing & Documentation	1 week

Conclusion: Achievable within semester timeline.

4.5 Legal Feasibility

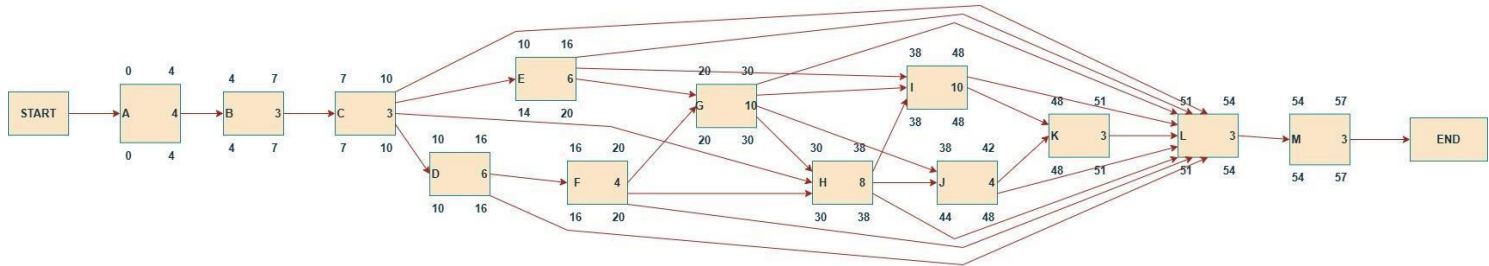
- No violation of data protection regulations.
- Tokens represent internal virtual currency.
- The system complies with university academic policies.
- Admin ensures ethical usage.

Conclusion: Legally feasible

CPM and PERT

Part 1: Critical Path Method (CPM)

1.1 CPM Network Diagram



1.1 CPM Table

ES = Early Start | EF = Early Finish
 LS = Late Start | LF = Late Finish
 TF = Total Float | FF = Free Float
 CP = Critical Path

Activity	Description	Dependency	Duration
A	Feasibility Analysis	—	4
B	Requirement Gathering	A	3
C	Requirement Analysis	B	3
D	System Design	C	6
E	Database Design	C	6
F	UI/UX	D	4
G	Implementing User Module Features	E, F	10
H	Implementing Booking Module	F, G, C	8
I	Implementing Wallet & Escrow	E, G, H	10
J	Implementing Rating Feature	G, H	4
K	Implementing Admin Control	I, J	3

L	Testing	C, D, E, F, G, H, I, J, K	3
---	---------	---------------------------	---

Activity	Description	Dependency	Duration
M	Documentation	L	3

1.2 CPM Total Float & Free Float

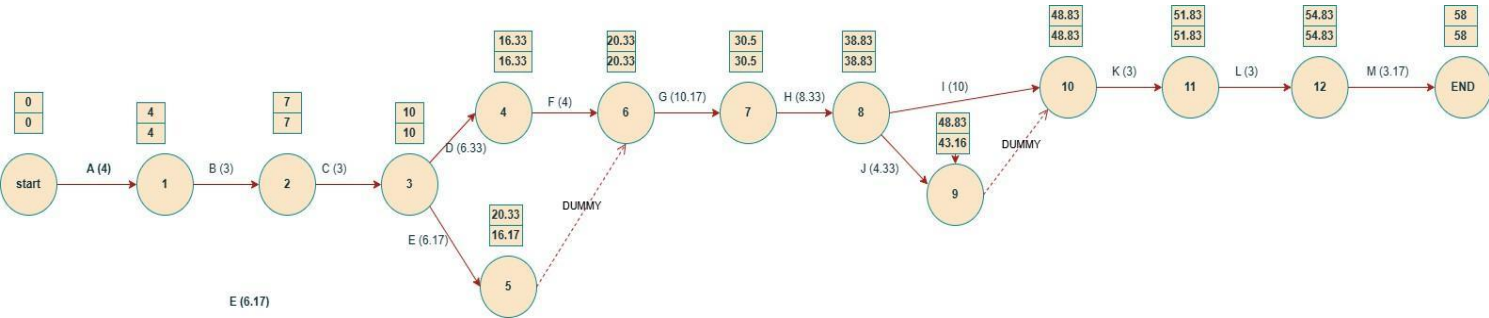
Activity	ES	EF	LS	LF	Total Float	Free Float	CP
A	0	4	0	4	0	0	✓
B	4	7	4	7	0	0	✓
C	7	10	7	10	0	0	✓
D	10	16	10	16	0	0	✓
E	10	16	14	20	4	4	—
F	16	20	16	20	0	0	✓
G	20	30	20	30	0	0	✓
H	30	38	30	38	0	0	✓
I	38	48	38	48	0	0	✓
J	38	42	44	48	6	6	—
K	48	51	48	51	0	0	✓
L	51	54	51	54	0	0	✓
M	54	57	54	57	0	0	✓

1.2 CPM Summary

Metric	Value
Project Length (Duration)	57 Days
Critical Path	A → B → C → D → F → G → H → I → K → L → M
Number of Critical Activities	11 out of 13
Non-Critical Activities	E, J

Part 2: Program Evaluation & Review Technique (PERT)

2.1 PERT Network Diagram



2.2 PERT Table

O = Optimistic | M = Most Likely (original) | P = Pessimistic

te = Expected Time = (O + 4M + P) / 6

σ² = Variance = ((P-O)/6)²

Activity	ID	Dependency	O	M	P	te	σ (Std Dev)
Feasibility Analysis	A	—	2	4	6	4.00	0.67

Activity	ID	Dependency	O	M	P	te	σ (Std Dev)
Requirement Gathering	B	A	1	3	5	3.00	0.67
Requirement Analysis	C	B	1	3	5	3.00	0.67
System Design	D	C	4	6	10	6.33	1.00
Database Design	E	C	4	6	9	6.17	0.83
UI/UX	F	D	2	4	6	4.00	0.67
Implementing User Module Features	G	E, F	7	10	14	10.17	1.17
Implementing Booking Module	H	F, G, C	6	8	12	8.33	1.00
Implementing Wallet & Escrow	I	E, G, H	6	10	14	10.00	1.33
Implementing Rating Feature	J	G, H	2	4	8	4.33	1.00
Implementing Admin Control	K	I, J	2	3	4	3.00	0.33
Testing	L	C, D, E, F, G, H, I, J, K	1	3	5	3.00	0.67
Documentation	M	L	2	3	5	3.17	0.50

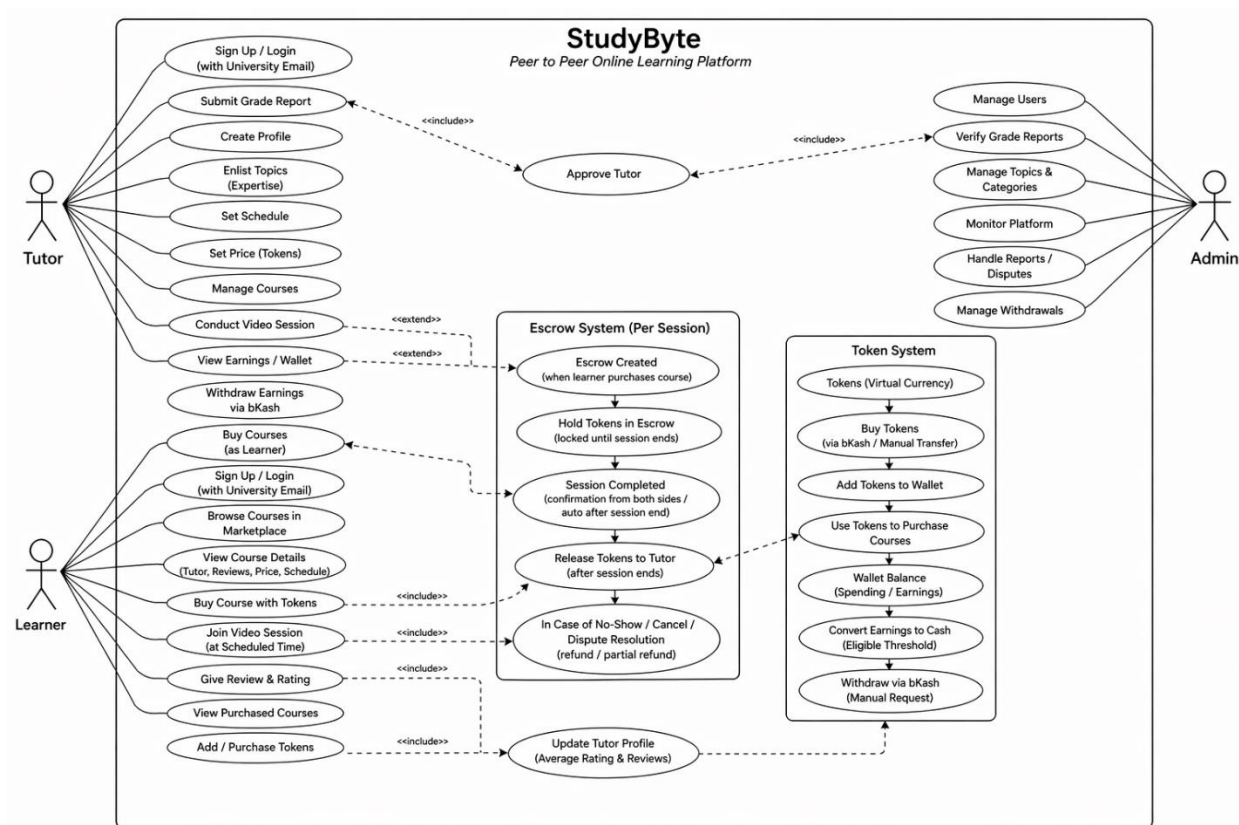
2.2 PERT Summary

Metric	Value
PERT Critical Path	A → B → C → D → F → G → H → I → K → L → M
Expected Project Length (ΣTE)	58 Days
Sum of Variances (σ^2) on Critical Path	8.20 days ²

Metric	Value
Standard Deviation (σ)	2.86 days

Diagrams

Use-Case Diagram:



System Actors:

- Tutor:** Provides courses and teaches students
- Learner:** Uses the platform to learn and purchase courses
- Admin:** Controls system operations and management

1. Tutor Functions:

- a. Login / Sign up using university email
- b. Submit grade report for verification
- c. Create and manage profile
- d. Add expertise topics
- e. Set available schedule
- f. Set course price (tokens)
- g. Manage courses (create, update, delete)
- h. Conduct video sessions
- i. View earnings and wallet
- j. Withdraw earnings via bKash

2. Learner Functions:

- a. Login / Sign up using university email
- b. Browse courses in marketplace
- c. View course details (tutor info, reviews, price, schedule)
- d. Buy course using tokens
- e. Join video session at scheduled time
- f. Give review and rating
- g. View purchased courses
- h. Add / purchase tokens

3. Admin Functions:

- a. Manage users (tutor and learner)
- b. Verify grade reports
- c. Approve tutors
- d. Manage topics and categories
- e. Monitor system activities
- f. Handle reports and disputes
- g. Manage withdrawal requests

System Modules Explanation

1. Escrow System (Per Session):

This system ensures secure payment between learner and tutor.

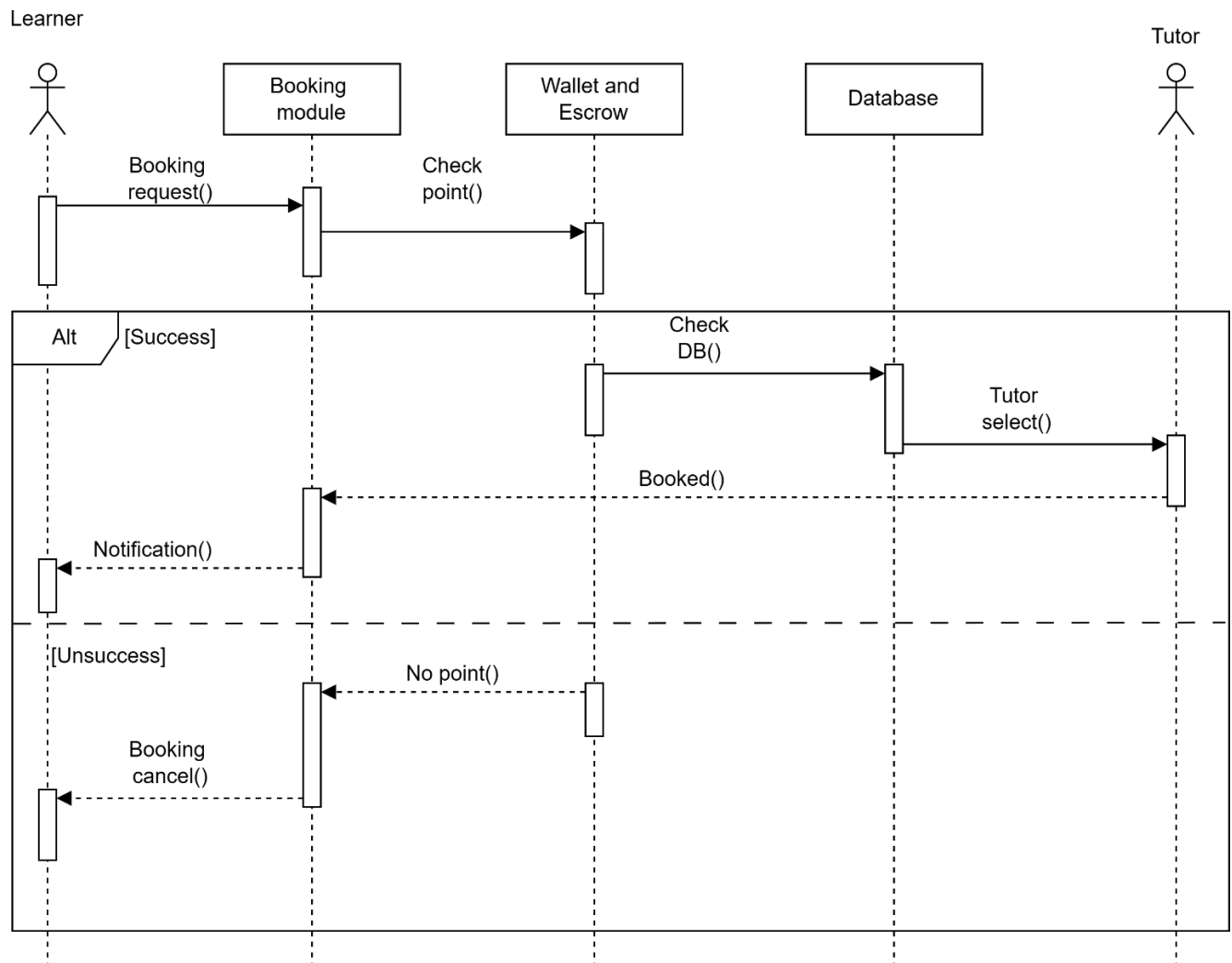
- When a learner buys a course, tokens are placed in escrow
- Tokens are held during the session
- After session completion:
 - Tokens are released to the tutor
- If there is a problem (cancel / no-show / dispute):
 - Refund or partial refund is given

2. Token System:

This system manages virtual currency.

- Tokens are used as payment
- Learners buy tokens via bKash or manual transfer
- Tokens are added to wallet
- Tokens are used to purchase courses
- Wallet shows spending and earnings
- Tutors convert tokens to cash
- Withdrawal is processed via bKash

Diagram: Sequence Diagram (Booking System)

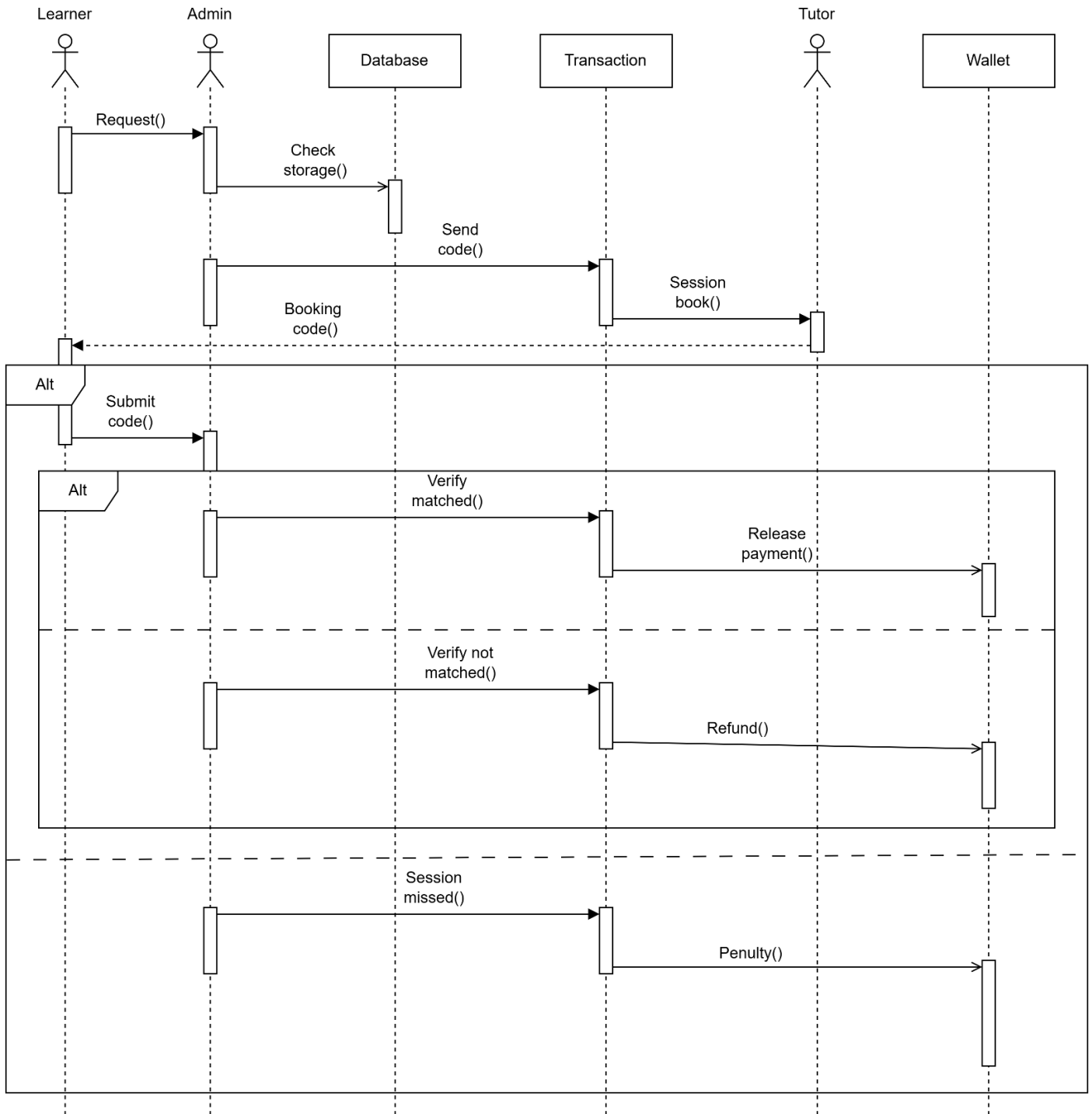


Explanation:

1. The learner opens the course page.
2. The learner selects a course and clicks “Book / Buy Course.”
3. The system checks if the learner is logged in.
4. If not logged in:
 - o The learner is asked to login or register first.
5. After login, the request is sent to the Booking Controller.
6. The system checks:
 - o Course availability
 - o Tutor schedule

7. If everything is valid:
 - The system creates a booking record in the database.
8. Then the Escrow system is activated:
 - Tokens are reserved (but not sent to tutor yet).
9. The system shows:
 - Booking confirmation message
 - Session time and details
10. If any error occurs (course not available / schedule conflict): The system shows an error message

Sequence Diagram (payment System)



Explanation:

1. The learner clicks “Buy Course” or “Buy Tokens.”
2. The learner selects the payment method (bKash / manual transfer).
3. The payment request is sent to the Payment Controller.
4. The system verifies:
 - o Payment details
 - o User wallet / token balance

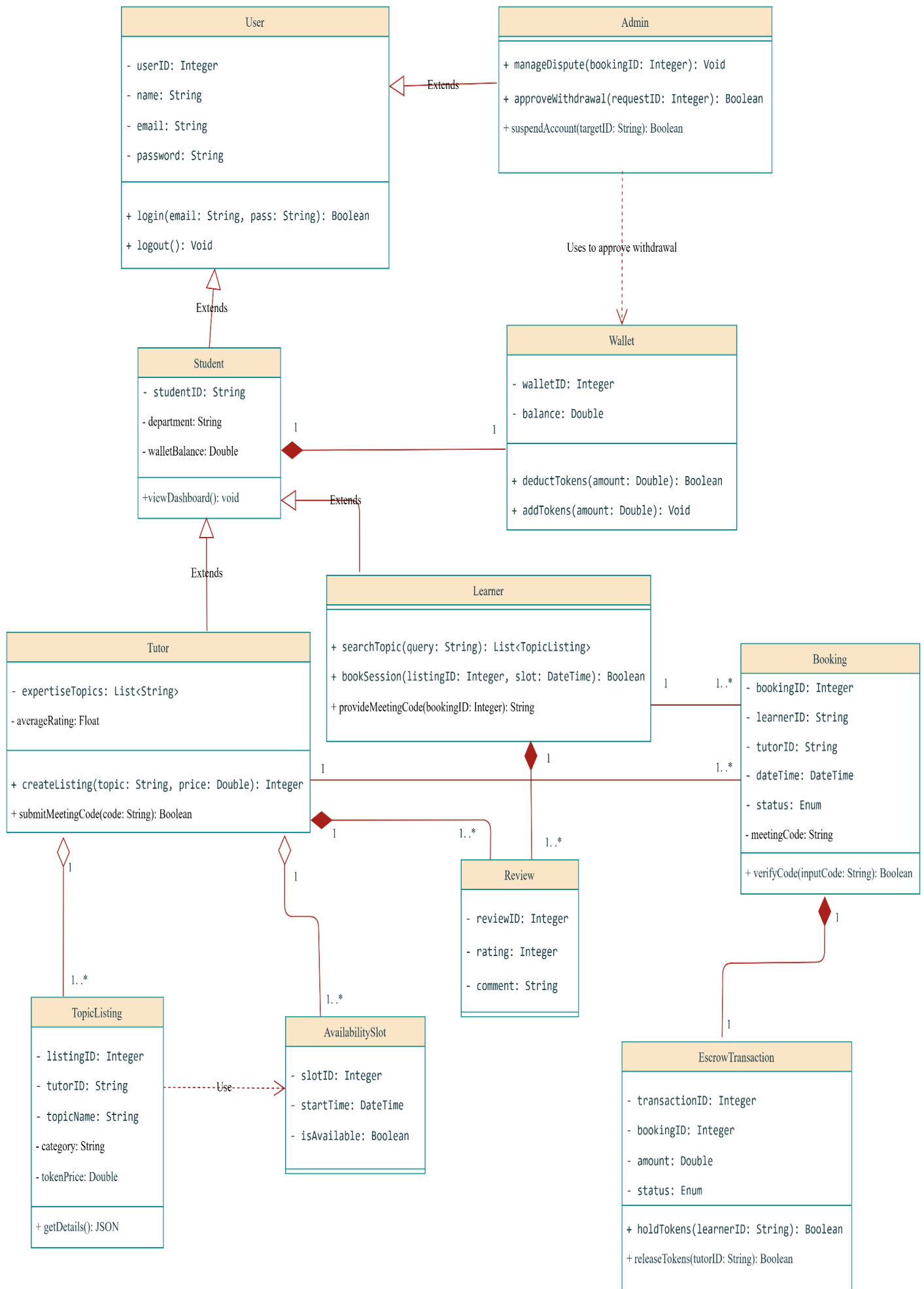
Case 1: Buying Tokens

5. If buying tokens:
 - o Payment is processed
 - o Tokens are added to learner wallet
6. The system shows:
 - o “Payment Successful”
 - o Updated wallet balance

Case 2: Booking Course (Using Tokens)

5. If purchasing course:
 - o Tokens are deducted from learner wallet
 - o Tokens are sent to Escrow system
6. Escrow system holds tokens until:
 - o Session is completed
7. After session completion:
 - o Tokens are released to tutor wallet
8. If there is an issue (cancel / dispute):
 - o Tokens are refunded or partially refunded
9. The system shows:
 - o Payment confirmation
 - o Transaction details

Diagram: UML Class Diagram



1. User (Base Class)

The **User** is the main class of the system.

- Stores common data:
 - Name
 - Email
 - Password
- A user can be:
 - **Tutor**
 - **Learner**
 - **Admin**

2. Tutor Class

The **Tutor** is a specialized user.

- Can:
 - Create courses
 - Set price and schedule
 - Conduct sessions
 - Receive payments
- Tutor must be **approved by admin**

3. Learner Class

The **Learner** is also a user.

- Can:
 - Browse courses
 - Book sessions
 - Buy tokens
 - Give ratings and reviews

4. Admin Class

The **Admin** controls the system.

- Can:
 - Manage users
 - Approve tutors
 - Handle disputes
 - Manage withdrawals

5. Course Class

The **Course** stores course information:

- Title
- Description
- Price (tokens)
- Schedule
- A course is created by a **Tutor**
- A course can be purchased by many **Learners**

6. Booking / Session Class

This class manages session booking.

- Stores:
 - Session time
 - Status (pending, completed, canceled)
- Links:
 - Learner
 - Tutor
 - Course

7. Token / Wallet Class

This class handles virtual money.

- Stores:
 - Token balance
 - Transactions
- Used by both:
 - Learner (spending)
 - Tutor (earning)

8. Escrow Class

The **Escrow system** ensures safe payment.

- Holds tokens during session
- Releases tokens after completion
- Handles refunds if needed

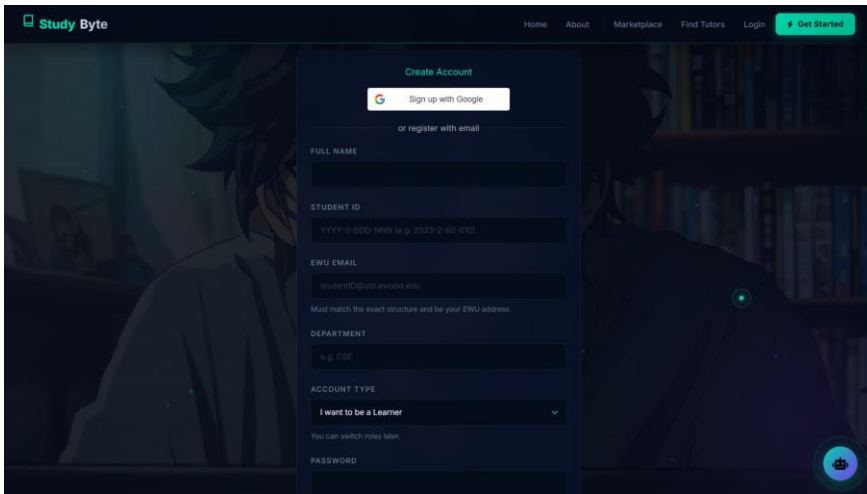
9. Review Class

This class stores ratings and feedback.

- Given by learner
- Linked to tutor and course

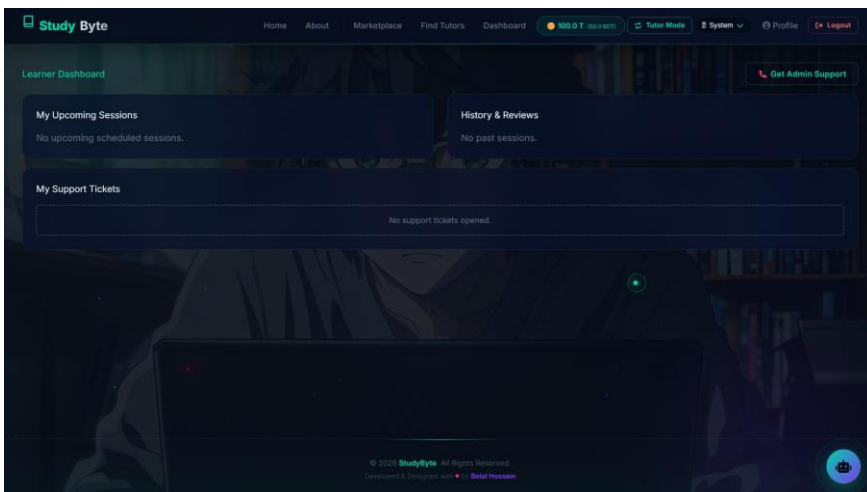
Screenshots

Register Page:



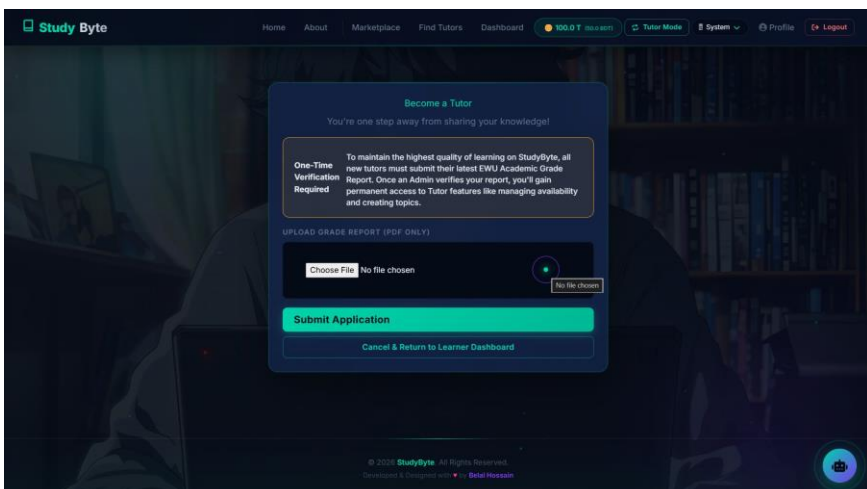
The screenshot shows the 'Create Account' form on the Study Byte website. The form is centered on a dark background with a faint illustration of a person. It includes a 'Sign up with Google' button, a 'or register with email' option, and several input fields: 'FULL NAME', 'STUDENT ID' (with a placeholder 'YYYY-S-DDD-MMM (e.g. 2023-2-60-010)'), 'EWU EMAIL' (with a placeholder 'student@gmail.edu' and a note 'Must match the exact structure and be your EWU address.'), 'DEPARTMENT' (with a placeholder 'e.g. CSE'), 'ACCOUNT TYPE' (a dropdown menu set to 'I want to be a Learner' with a note 'You can switch roles later.'), and 'PASSWORD'. A 'Get Started' button is in the top right corner of the page.

Learner Dashboard:



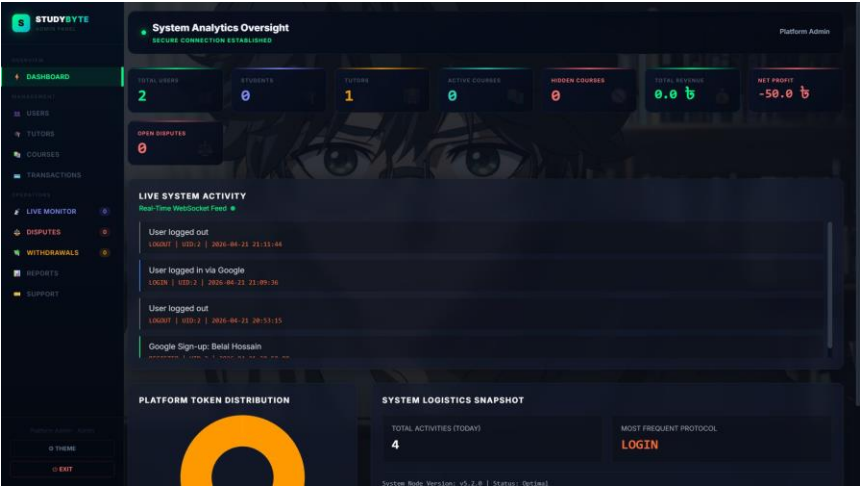
The screenshot shows the 'Learner Dashboard' on the Study Byte website. The dashboard has a dark theme with a faint background illustration. It features a 'Get Admin Support' button in the top right. Below the header, there are three main sections: 'My Upcoming Sessions' (showing 'No upcoming scheduled sessions.'), 'History & Reviews' (showing 'No past sessions.'), and 'My Support Tickets' (showing 'No support tickets opened.'). At the bottom, there is a footer with copyright information: '© 2020 StudyByte. All Rights Reserved. Developed & Designed with ❤️ by Bilal Hossain'.

Become a Tutor:

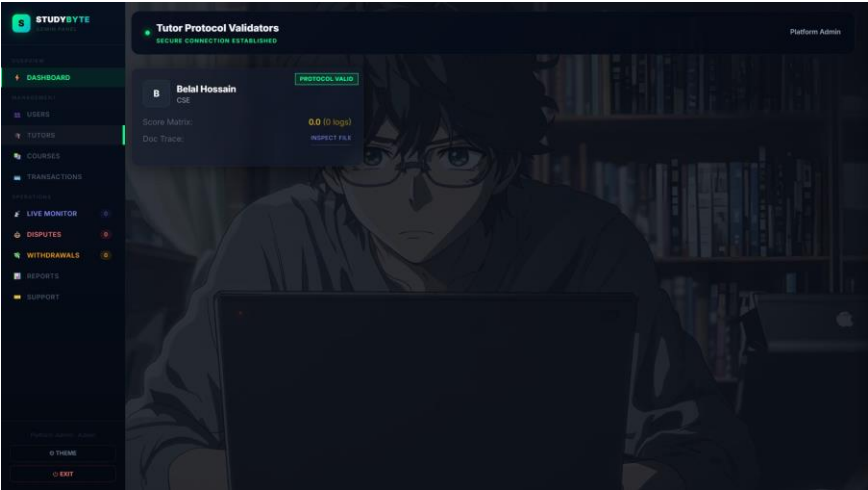


The screenshot shows the 'Become a Tutor' form on the Study Byte website. The form is a modal dialog box with a dark background and a faint illustration. It includes a 'You're one step away from sharing your knowledge!' message. Below this, there is a 'One-Time Verification Required' section with a note: 'To maintain the highest quality of learning on StudyByte, all new tutors must submit their latest EWU Academic Grade Report. Once an Admin verifies your report, you'll gain permanent access to Tutor features like managing availability and creating topics.' There is an 'UPLOAD GRADE REPORT (PDF ONLY)' section with a 'Choose File' button and a 'No file chosen' message. At the bottom, there are two buttons: 'Submit Application' (in green) and 'Cancel & Return to Learner Dashboard'.

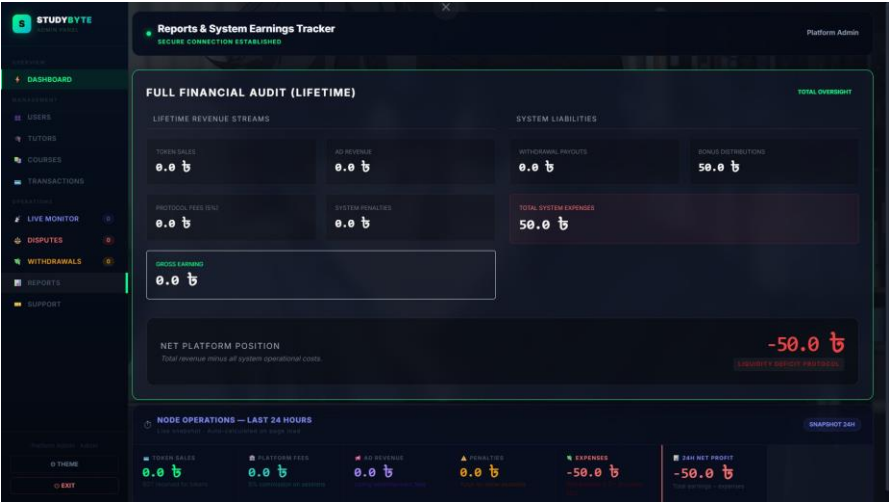
Admin Dashboard:



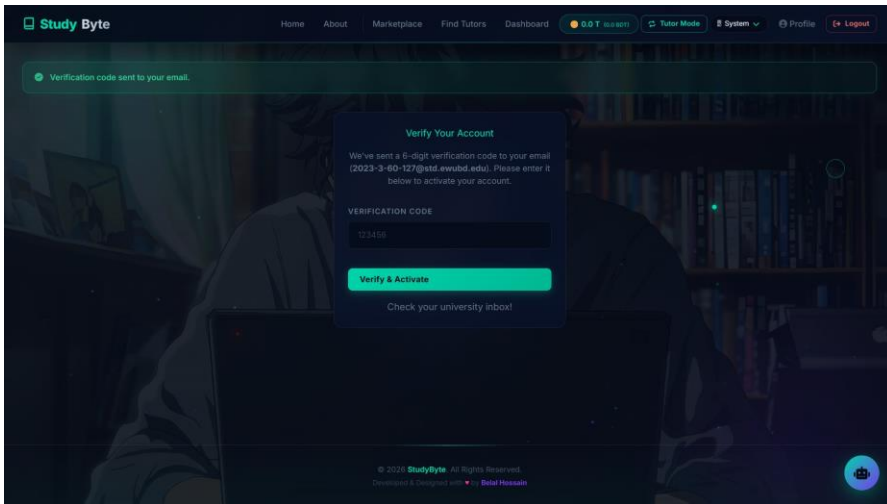
Admin - Tutor Validators:



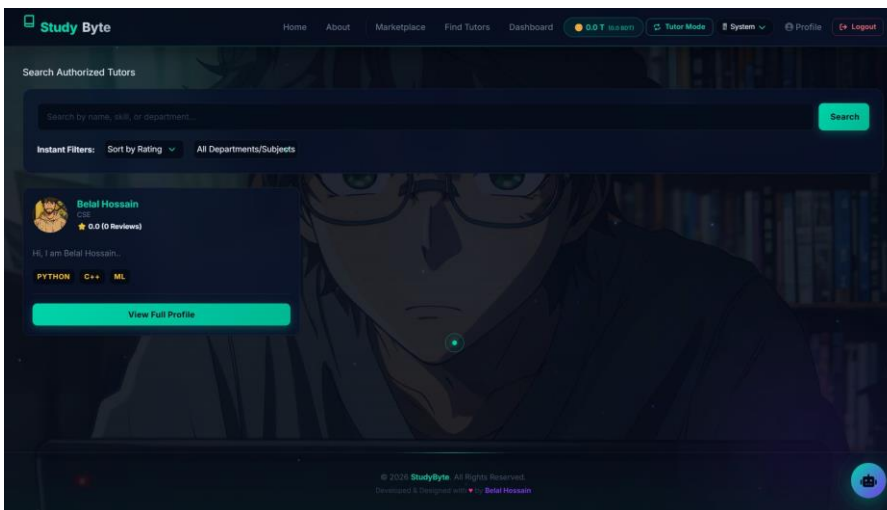
Admin - Financial Reports:



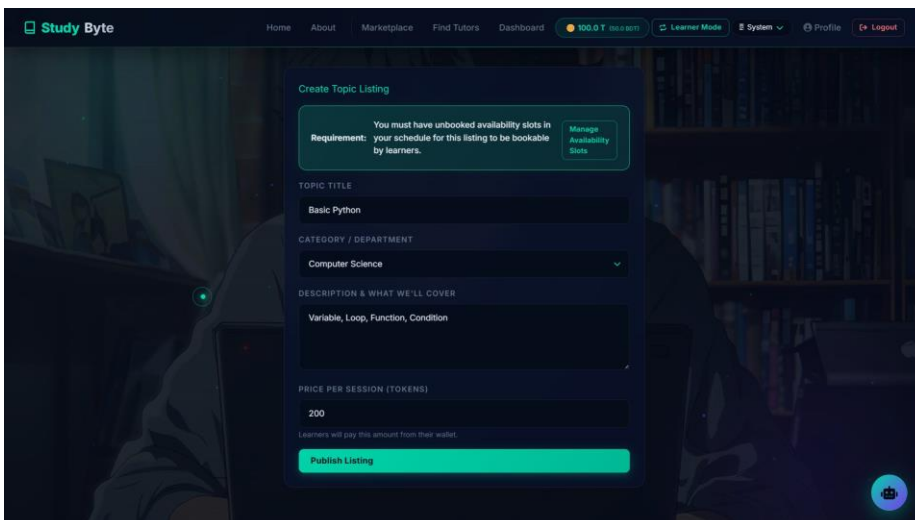
Email Verification:



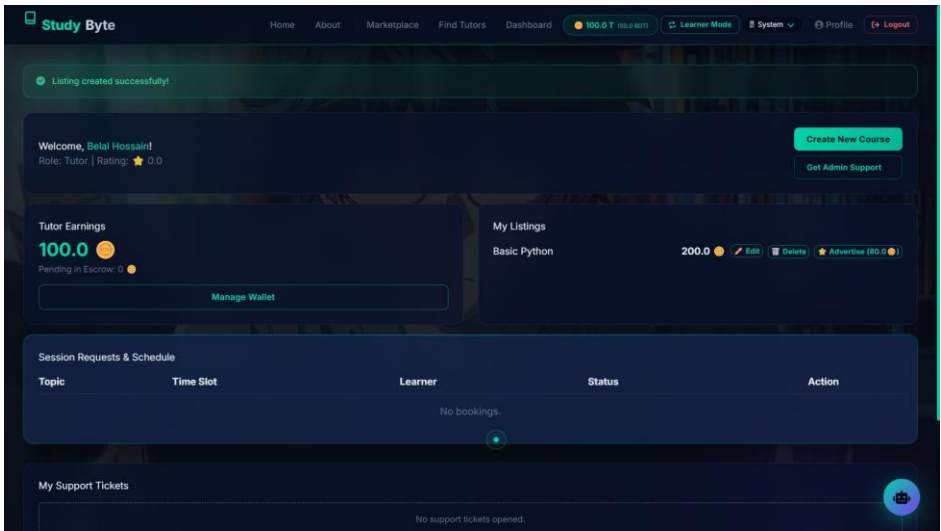
Find Tutors:



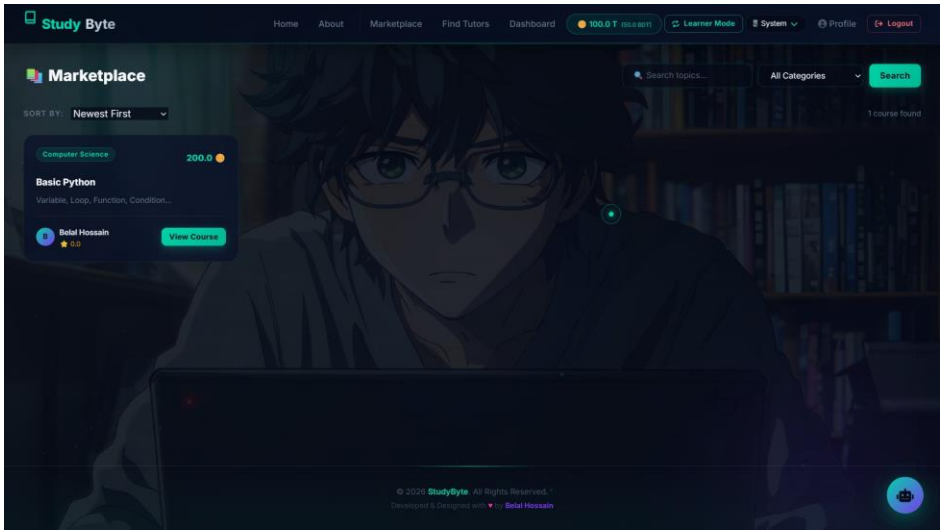
Create Topic Listing:



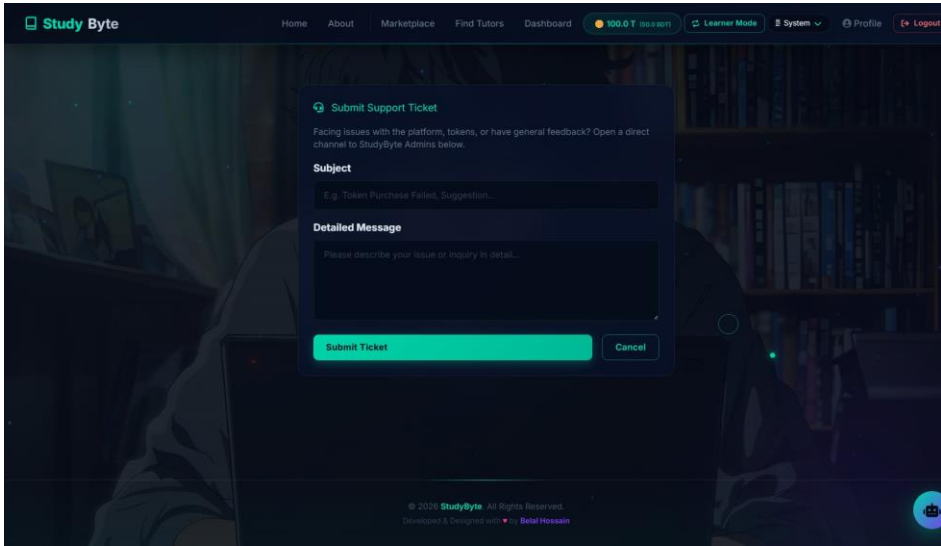
Tutor Dashboard:



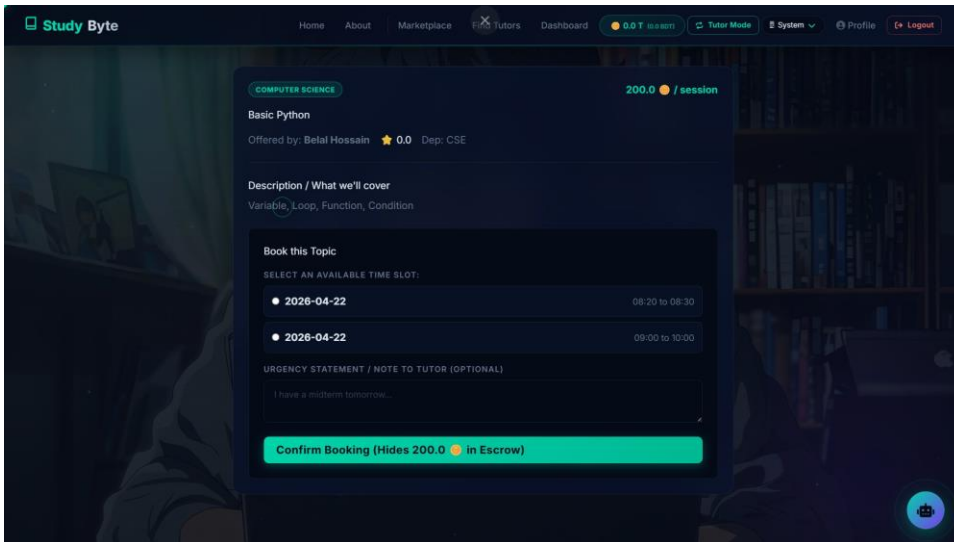
Marketplace:



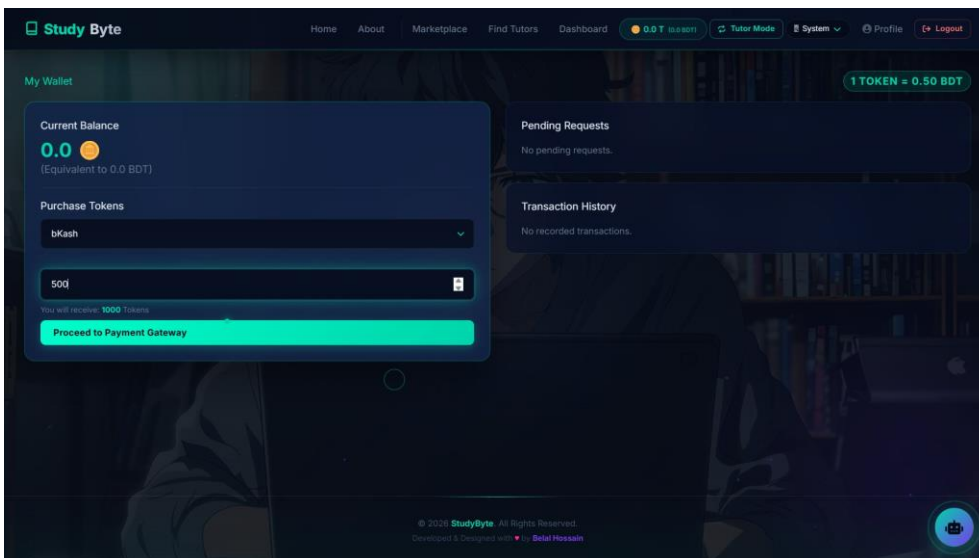
Support Ticket:



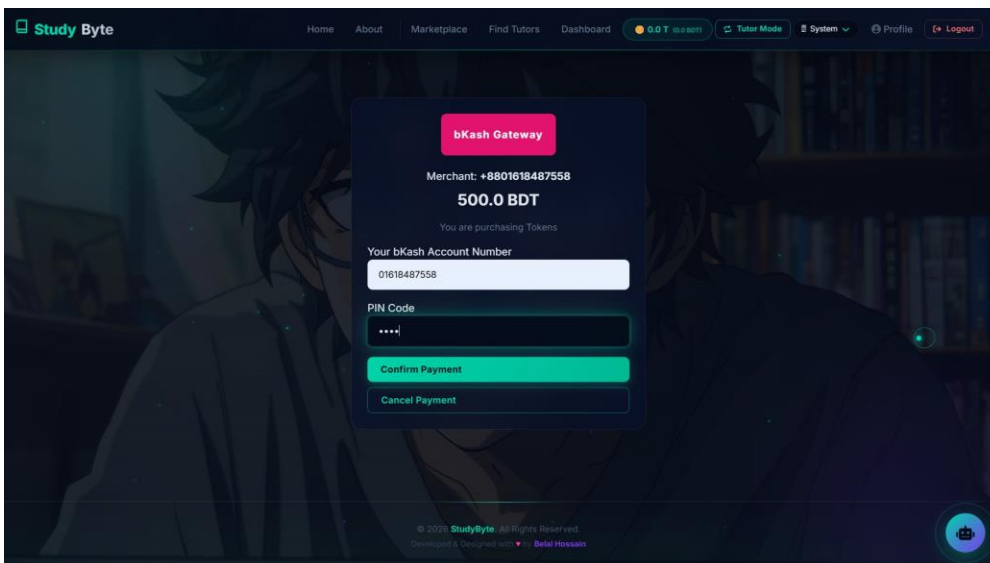
Course Booking:



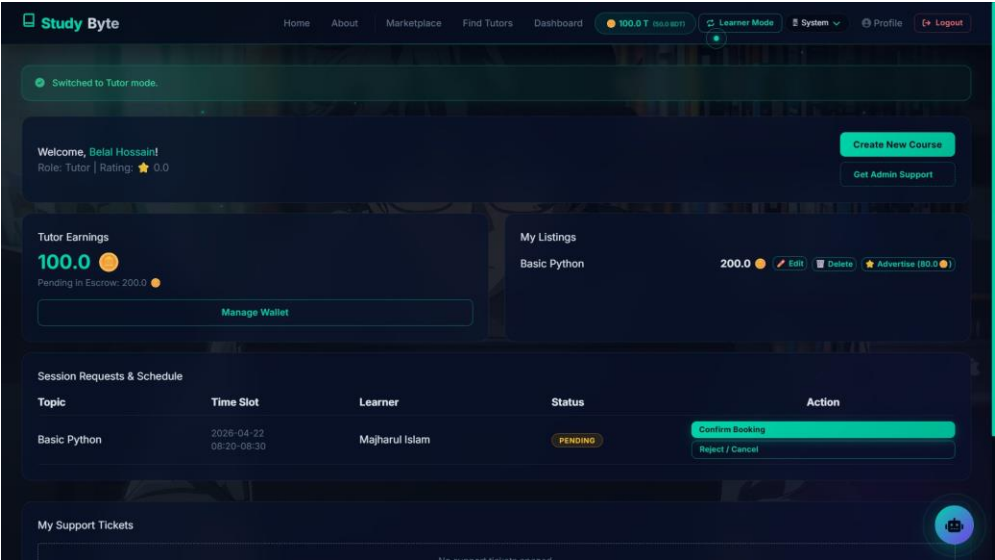
Wallet:



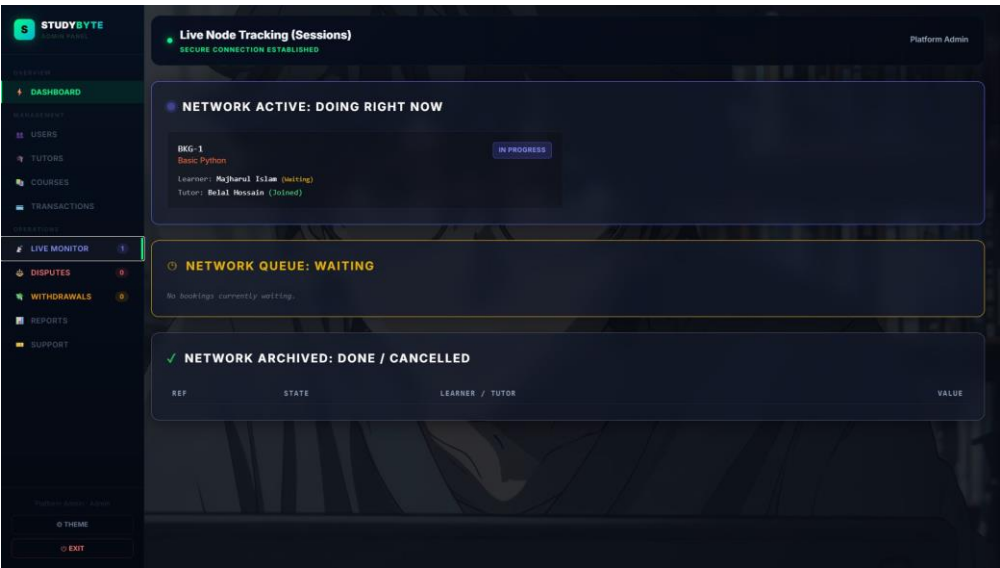
bKash Payment Gateway:



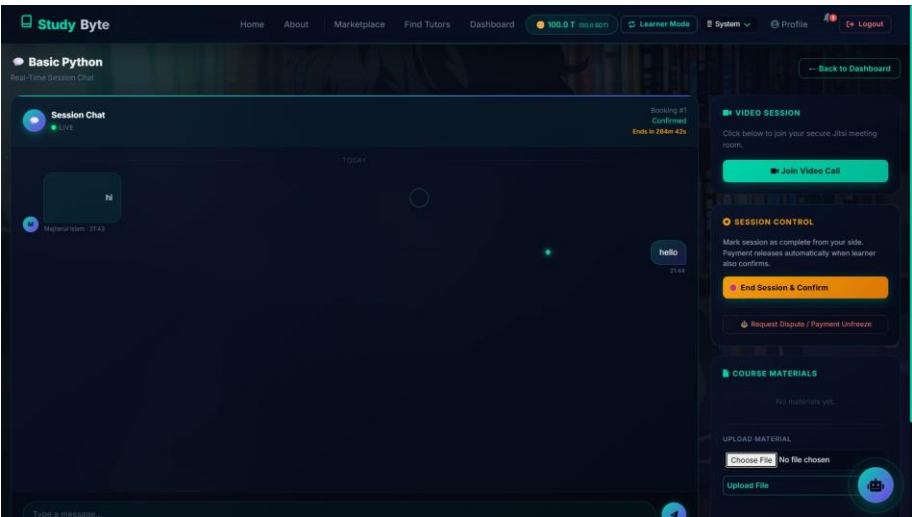
Tutor Session Request:



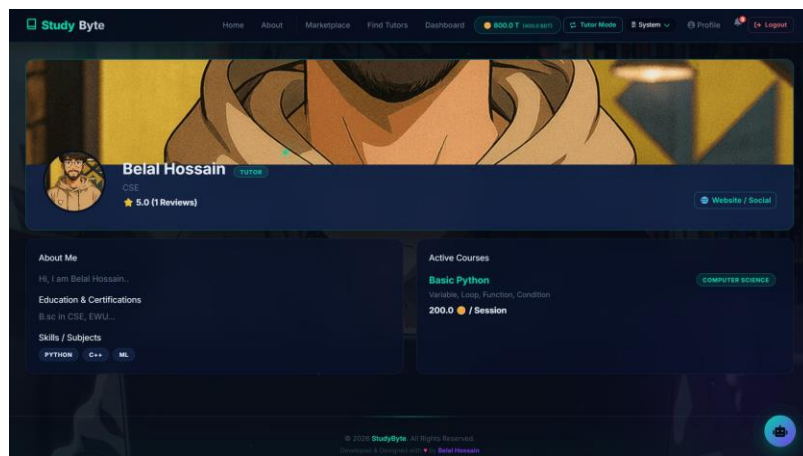
Admin - Live Monitor:



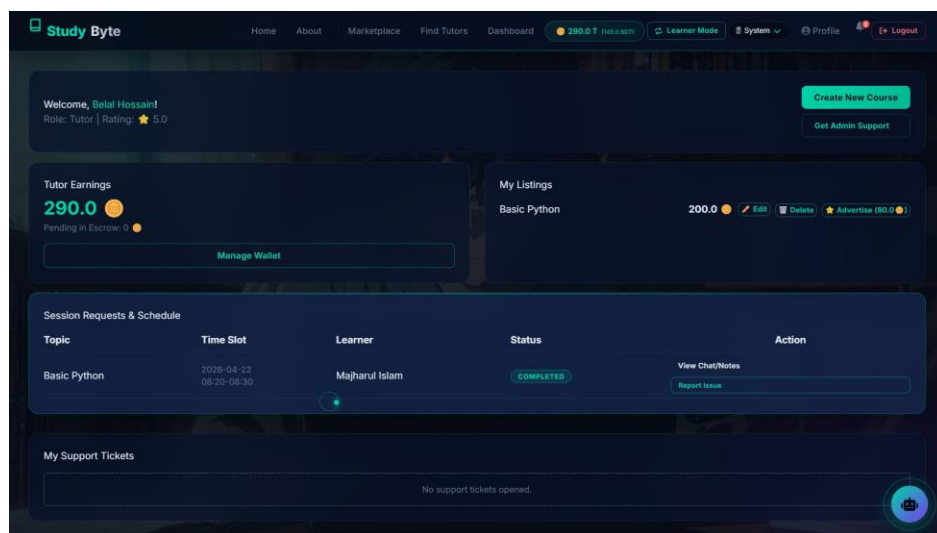
Session Chat:



Tutor Profile Page:



Tutor Dashboard with Earnings and Session History:



Video Session:



Dispute / Unfreeze Request Form:

Study Byte

[Home](#)[About](#)[Marketplace](#)[Find Tutors](#)[Dashboard](#)

800.0 T (wallet)

[Tutor Mode](#)

System

[Profile](#)[Logout](#)

Request Dispute / Unfreeze

BKG-2 / Basic Python

If the session ended but payment is frozen, or if there was an issue, submit a dispute. Admin will review and take action on the escrow. If both parties confirm the session was completed, payment is released automatically — no dispute needed.

DISPUTE CATEGORY

Select the issue type

Payment / Escrow

Request payment unfreeze (session ended, no auto-release)

Payment stuck in escrow after both confirmed

No Show / Misconduct

Tutor did not show up for the session

Learner did not show up for the session

Session was cancelled unfairly

Quality / Content

Session was incomplete or unsatisfactory

Tutor taught different content than advertised

Other

Other

© 2026 StudyByte. All Rights Reserved.

Admin Dispute Resolution Panel:

Study Byte

[Home](#)[About](#)[Marketplace](#)[Find Tutors](#)

Admin

System

[Profile](#)

Logout

Dispute resolved with action: penalty

Dispute Resolution

Manage escrow decisions and cancel invalid disputes.

Admin

DISPUTES OPEN

0

SYSTEM PROFIT

955.0

TOTAL REVENUE

1005.0

TOTAL EXPENSES

50.0

Active Disputes

1 Total

BOOKING	REPORTER	ISSUE	ESCROW	STATUS	ADMIN ACTION
BKG-2 Basic Python 2026-04-22 09:00	Nusrat Jahan Learner 24 Apr 2026 22:36	Tutor No Show Wait but tutor did not show	No escrow held	RESOLVED	Resolved

© 2026 StudyByte. All Rights Reserved.

Designed & Developed with by Bilal Hossain

Admin Financial Reports and Earnings Tracker:

STUDYBYTE

Reports & System Earnings Tracker

Platform Admin

Dashboard

Management

Users

Tutors

Courses

Transactions

Open Orders

Live Monitor

Disputes

Withdrawals

Reports

Support

Reports & System Earnings Tracker

Secure Connection Established

Platform Admin

FULL FINANCIAL AUDIT (LIFETIME)

TOTAL OVERSIGHT

LIFETIME REVENUE STREAMS

SYSTEM LIABILITIES

TOKEN SALES

1000.0 T

AD REVENUE

0.0 T

WITHDRAWAL REQUESTS

0.0 T

BONUS DISTRIBUTIONS

50.0 T

PROTOCOL FEE (5%)

5.0 T

SYSTEM PENALTIES

20.0 T

TOTAL SYSTEM EXPENSES

50.0 T

GROSS EARNED

1025.0 T

NET PLATFORM POSITION

+975.0 T

Total revenue minus all system operational costs.

PROFITABLE, PROUD, STRATEGIC

NODE OPERATIONS — LAST 24 HOURS

SNAPSHOT 24H

TOKEN SALES

1000.0 T

PLATFORM FEES

5.0 T

AD REVENUE

0.0 T

PENALTIES

20.0 T

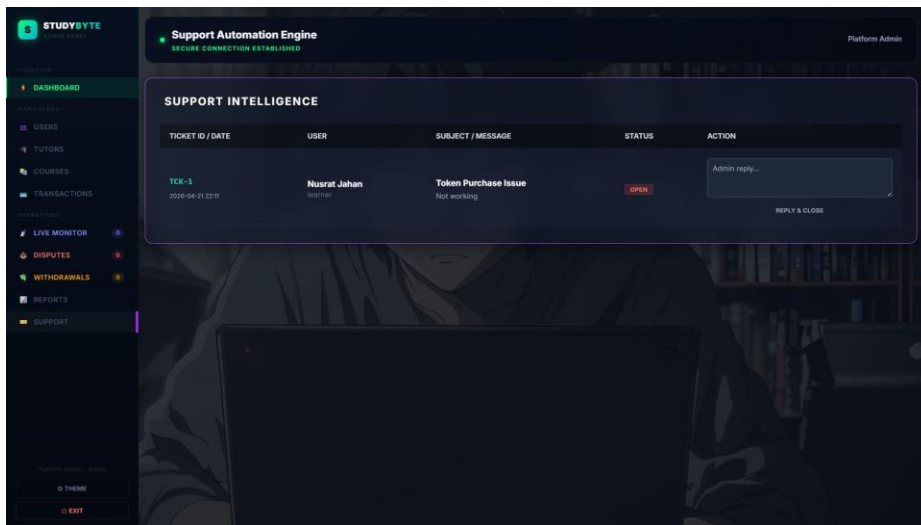
EXPENSES

-50.0 T

24H NET PROFIT

+975.0 T

Admin Support Intelligence Panel:



Conclusion

StudyByte has been successfully developed as a functional peer-to-peer micro-tutoring platform that bridges the gap between university students seeking help and those willing to share their knowledge. Throughout the project lifecycle, the team designed and implemented a comprehensive system encompassing role-based access control, a digital token economy with escrow protection, real-time session management, dispute resolution, and a full-featured administrative panel.

The screenshots presented in this report demonstrate the live working state of the platform, covering key features such as the Jitsi-powered video sessions, real-time session chat with course material sharing, tutor profiles and dashboards, the dispute and unfreeze request workflow, and the admin panels for financial oversight and support management. Together, these features validate that StudyByte achieves its core objectives of security, transparency, and usability. From a technical standpoint, the integration of the escrow system ensures that neither the tutor nor the learner can exploit the platform, as payment is only released upon mutual session confirmation. The admin dispute resolution module further strengthens trust by providing a fair mechanism to handle conflicts. The financial reporting dashboard allows the platform operator to maintain full visibility into token flows, protocol fees, and system penalties.

In conclusion, StudyByte stands as a complete and deployable information system that meets all stated functional and non-functional requirements. The project not only demonstrates sound software engineering practices but also reflects a real-world approach to solving the problem of accessible, affordable, and accountable peer tutoring. Future enhancements could include built-in video conferencing, AI-based tutor recommendation, multi-institution support, and a mobile application to further extend the platform's reach and impact.